Benemérita Universidad Autónoma de Puebla - date

Facultad de Ciencias de la Computación

Ingeniería en Tecnologías de la Información

AGRUPAMIENTO JERÁRQUICO POR EL MÉTODO DE WARD

Docente Pedro Tecanhuehue Vera

Hecho por:
Adylene Baylon Cuahtlapantzi
Karla Irais De Florencio Romero
Maria Fernanda Flores Ramos

30 de abril de 2026

Índice

1. Descripción de la base de datos **3**
- 1.1. Descripción general de los atributos . . . 3
- 1.2. Tipo de datos . . . 3
- 1.3. Importancia del dataset . . . 3

2. Método de WARD **3**
- 2.1. Idea principal . . . 4
- 2.2. Cálculo del costo de fusión . . . 4
- 2.3. Centroide . . . 4
- 2.4. Procedimiento del algoritmo . . . 5
- 2.5. Dendrograma . . . 5
- 2.6. Características . . . 5

3. Métodología **5**

4. Pruebas **6**
- 4.1. Dendrograma . . . 6
- 4.2. Asiganción de objetos . . . 8
- 4.3. Matriz de confusión . . . 9

5. Conclusiones **10**

6. APENDICES **11**
- 6.1. Descripción de código . . . 11
  - 6.1.1. Lectura de datos . . . 11
  - 6.1.2. Normalización . . . 13
  - 6.1.3. Distancia euclidiana . . . 14
  - 6.1.4. Centroide . . . 14
  - 6.1.5. Costo de Ward . . . 15
  - 6.1.6. Clustering jerárquico . . . 16
  - 6.1.7. Obtención de clusters . . . 17
  - 6.1.8. Matriz de confusión . . . 18
  - 6.1.9. Dendrograma . . . 20
  - 6.1.10. Ejecución principal . . . 21
- 6.2. Conceptos usados . . . 21
- 6.3. Manual de usuario . . . 22
  - 6.3.1. 1. Preparación del entorno . . . 22
  - 6.3.2. 2. Organización de archivos . . . 22
  - 6.3.3. 3. Ejecución del programa . . . 23
  - 6.3.4. 4. Resultados generados . . . 24
  - 6.3.5. 5. Interpretación de resultados . . . 25
  - 6.3.6. 6. Recomendaciones . . . 26
- 6.4. Manual técnico . . . 26

6.5. Referencias 26

1. Descripción de la base de datos

La base de datos utilizada corresponde al conjunto *Higher Education Students Performance Evaluation*, disponible en el repositorio de UCI. Este conjunto fue recolectado en 2019 a partir de encuestas aplicadas a estudiantes de educación superior, con el objetivo de analizar y predecir su desempeño académico.
El dataset está compuesto por aproximadamente 145 registros y más de 30 atributos, todos de tipo categórico, los cuales describen características personales, familiares y hábitos académicos de los estudiantes.

1.1. Descripción general de los atributos

Los atributos pueden agruparse en tres grandes categorías:
1. Características personales: Incluyen información básica del estudiante, como edad, sexo, tipo de escuela de procedencia, tipo de beca, si trabaja adicionalmente, participación en actividades extracurriculares, estado sentimental, nivel de ingresos y tipo de transporte.
2. Entorno familiar: Consideran aspectos como el nivel educativo de los padres, número de hermanos, estado civil de los padres y ocupación de los mismos. Estas variables permiten analizar el contexto socioeconómico del estudiante.
3. Hábitos y comportamiento académico: Incluyen factores relacionados con el estudio, como asistencia a clases, preparación para exámenes, hábitos de estudio, uso de recursos educativos y otras actividades académicas.
Finalmente, el conjunto incluye una variable objetivo denominada **GRADE**, la cual representa el desempeño académico del estudiante y es utilizada como clase real para evaluar los resultados del clustering.

1.2. Tipo de datos

Todos los atributos del dataset son categóricos y se encuentran codificados numéricamente, donde cada número representa una categoría específica (por ejemplo, rangos de edad, tipos de beca o niveles educativos). Esto permite aplicar algoritmos de análisis sin necesidad de transformaciones complejas.

1.3. Importancia del dataset

Este conjunto de datos es ampliamente utilizado en problemas de minería de datos y aprendizaje automático, ya que permite analizar los factores que influyen en el rendimiento académico y detectar patrones en el comportamiento de los estudiantes. En este trabajo, se utilizó para aplicar el método de Ward y analizar la formación de grupos en función de la similitud entre los datos.

2. Método de WARD

El método de Ward es una técnica de *clustering jerárquico aglomerativo* cuyo objetivo es agrupar objetos minimizando la varianza dentro de cada grupo. A diferencia de otros métodos, Ward no se basa únicamente en la distancia entre puntos, sino en el incremento de la suma de errores cuadrados (SSE) al fusionar clusters.

Inicialmente, cada objeto del conjunto de datos se considera como un cluster individual. En cada iteración, se seleccionan dos clusters cuya fusión produce el menor incremento en la varianza interna total.

2.1. Idea principal

El criterio de Ward busca minimizar la siguiente función:

$$SSE = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

donde:

- C_k es el cluster k
- μ_k es el centroide del cluster k
- x_i es un punto dentro del cluster

En cada paso, el algoritmo une los dos clusters que generan el menor aumento en esta suma.

2.2. Cálculo del costo de fusión

El costo de unir dos clusters C_i y C_j en el método de Ward se calcula como:

$$D(C_i, C_j) = \frac{n_i \cdot n_j}{n_i + n_j} |\mu_i - \mu_j|^2$$

donde:

- n_i, n_j son los tamaños de los clusters
- μ_i, μ_j son los centroides de los clusters
- $|\mu_i - \mu_j|$ es la distancia euclidiana entre centroides

2.3. Centroide

El centroide de un cluster se define como el promedio de sus elementos:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

2.4. Procedimiento del algoritmo

El algoritmo sigue los siguientes pasos:

1. Inicializar cada objeto como un cluster independiente.
2. Calcular el costo de fusión entre todos los pares de clusters.
3. Unir los dos clusters con menor costo.
4. Actualizar los centroides.
5. Repetir el proceso hasta obtener un solo cluster o el número deseado de grupos.

2.5. Dendrograma

El resultado del proceso se representa mediante un dendrograma, el cual muestra la secuencia de fusiones y las distancias a las que ocurren. Este permite seleccionar el número adecuado de clusters mediante una línea de corte.

2.6. Características

- Minimiza la varianza dentro de los clusters.
- Produce clusters compactos y bien definidos.
- Es sensible a la escala de los datos, por lo que se recomienda normalizar.
- No requiere especificar el número de clusters desde el inicio.

3. Métodología

Para resolver el problema de agrupamiento, se implementó el método de Ward desde cero, siguiendo una serie de etapas bien definidas que permitieron procesar los datos, aplicar el algoritmo y evaluar los resultados obtenidos.
En primer lugar, se realizó la lectura del archivo de datos en formato CSV, separando las variables numéricas (utilizadas como entrada del algoritmo) y la clase real de cada objeto, la cual se reservó únicamente para la evaluación posterior. Durante esta etapa, se aseguraron conversiones adecuadas a valores numéricos para evitar inconsistencias en el procesamiento.
Posteriormente, se aplicó una normalización de tipo Min-Max a las variables, con el objetivo de escalar todos los atributos al mismo rango y evitar que aquellos con mayor magnitud influyeran de manera desproporcionada en el cálculo de distancias. Esto es especialmente importante en métodos basados en distancia como Ward.
Una vez preparados los datos, se implementó el algoritmo de clustering jerárquico aglomerativo utilizando el criterio de Ward. Inicialmente, cada objeto se consideró como un cluster independiente. En cada iteración, se calcularon los costos de fusión entre todos los pares de clusters, seleccionando aquellos cuya unión minimizaba el incremento en la varianza interna. Este proceso se repitió hasta formar un único cluster, registrando en cada paso la información necesaria para construir el dendrograma.

Con base en el historial de fusiones, se generó el dendrograma, el cual permitió visualizar la estructura jerárquica de los datos. A partir de este, se analizaron diferentes niveles de corte con el fin de identificar una partición que se aproximara a ocho grupos, de acuerdo con las características del conjunto de datos. Posteriormente, se generaron agrupaciones específicas para distintos valores de k (3, 5 y 8 clusters), y se almacenaron los resultados en archivos CSV para facilitar su análisis. Finalmente, se evaluó la calidad del agrupamiento mediante una matriz de confusión, comparando los clusters obtenidos con las clases reales del conjunto de datos. Esta matriz permitió asignar un nombre a cada cluster en función de la clase predominante en cada uno.
En conjunto, esta metodología permitió no solo aplicar el algoritmo de Ward, sino también interpretar y validar los resultados obtenidos de manera estructurada.

4. Pruebas

Con el objetivo de evaluar el comportamiento del método de Ward, se realizaron diversas pruebas utilizando el conjunto de datos proporcionado. Estas pruebas permitieron analizar la formación de clusters, la estructura jerárquica de los datos y la relación entre los grupos obtenidos y las clases reales.

4.1. Dendrograma

En primer lugar, se generó el **dendrograma** a partir del historial de fusiones del algoritmo. Este permitió visualizar cómo los objetos se agrupan progresivamente en función de la distancia entre clusters. A través de la inspección visual del dendrograma, se identificaron distintos niveles de corte, observándose que en niveles altos se generan pocos grupos; como vemos en la figura 1 se hicieron 2 cortes en la parte de arriba generando 3 y 5 clusters, mientras que en niveles bajos se obtiene una mayor fragmentación de los datos, como vemos en la figura 2, donde obtenemos 8 clusters.

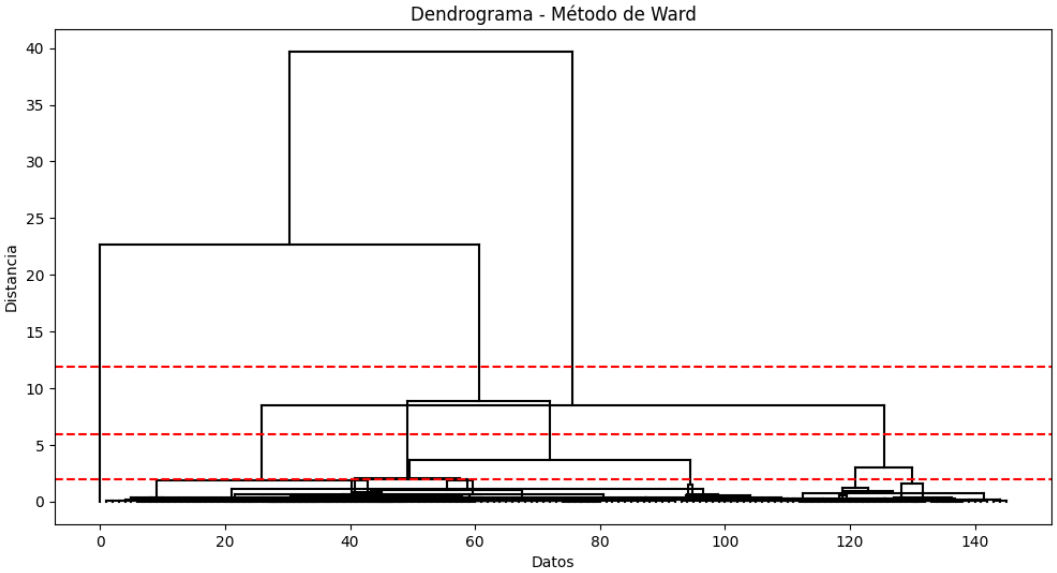

Figura 1: Dendrogama con 2 cortes muy arriba

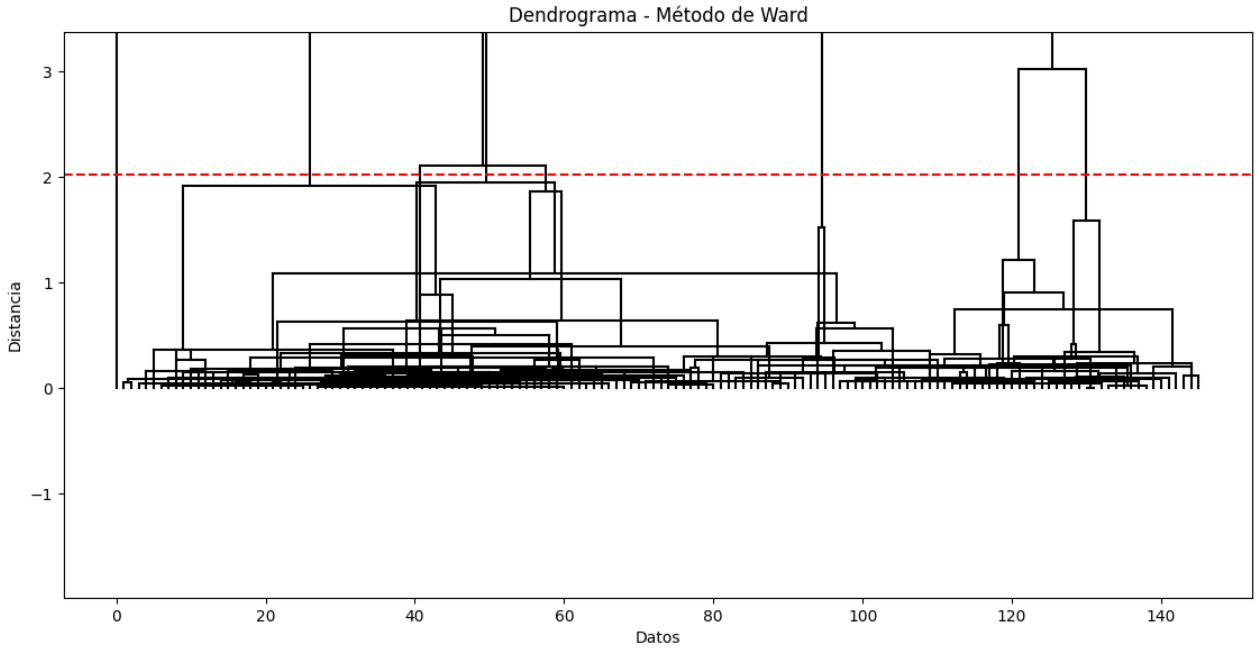

Figura 2: Zoom del dendrogama con el tercer corte, generando 8 clusters

Una vez que supimos dónde cortar y formar esos cortes visualmente, pasamos al código, que con ayuda de este, obtuvimos casi exactamente el número de clústers que no pide el profesor.

4.2. Asiganción de objetos

Posteriormente, se analizaron diferentes valores de k (3, 5 y 8 clusters), generando para cada caso un archivo CSV con la asignación de los objetos a sus respectivos grupos. Estos archivos permitieron observar cómo varía la distribución de los datos dependiendo del número de clusters seleccionado.
En la Figura 3 al archivo `clusters_8.csv`, se observa la asignación de cada objeto a uno de los ocho clusters generados por el algoritmo. Cada fila representa un objeto del conjunto de datos, donde las columnas contienen sus atributos normalizados y la última columna indica el cluster al que fue asignado (por ejemplo, `Cluster_1`, `Cluster_2`, etc.). Se puede apreciar que los datos se distribuyen entre los distintos clusters, lo que indica que el algoritmo logró segmentar el conjunto en múltiples grupos con características similares.

1	1	1	1	1	1	1	1	1	1	1	1	0	Cluster_1
0.05555556	0	0	0	0	0.04347826	0.08333333	0.04	0.03846154	0	0.07142857	0.03448276	0.11111111	Cluster_2
0	0	0	0.04761905	0	0	0.04166667	0.04	0.03846154	0.03703704	0	0	0.11111111	Cluster_2
0.05555556	0	0	0	0	0	0.04166667	0.04	0.03846154	0.07407407	0.10714286	0.03448276	0.11111111	Cluster_2
0.05555556	0	0	0	0.09090909	0	0.08333333	0.04	0.03846154	0	0	0	0.11111111	Cluster_2
0.11111111	0	0.1	0	0.09090909	0.04347826	0.08333333	0	0.07692308	0.07407407	0.10714286	0.06896552	0.11111111	Cluster_2
0	0	0	0	0	0	0.04166667	0.08	0.07692308	0.07407407	0.03571429	0.03448276	0.33333333	Cluster_2
0.05555556	0	0	0.04761905	0	0	0.08333333	0.04	0.07692308	0.07407407	0.10714286	0.06896552	0.11111111	Cluster_2
0.05555556	0.05263158	0	0.04761905	0	0	0.08333333	0	0.07692308	0.07407407	0.07142857	0.06896552	0.11111111	Cluster_2
0.05555556	0	0	0	0	0	0.08333333	0.04	0.03846154	0.03703704	0.10714286	0.06896552	0.11111111	Cluster_2
0.05555556	0	0	0.04761905	0	0	0.04166667	0.08	0.07692308	0	0.07142857	0.06896552	0.11111111	Cluster_2
0.05555556	0	0	0.04761905	0	0	0.04166667	0.04	0.07692308	0.03703704	0.10714286	0.06896552	0.33333333	Cluster_2
0.11111111	0	0	0.04761905	0	0	0.08333333	0.08	0.03846154	0.03703704	0.03571429	0.03448276	0.44444444	Cluster_2
0	0	0	0.04761905	0	0	0.08333333	0.04	0.07692308	0.07407407	0.10714286	0.03448276	0.11111111	Cluster_2
0.05555556	0	0	0	0.04545455	0	0.08333333	0.04	0.07692308	0	0.07142857	0.03448276	0.33333333	Cluster_2
0.05555556	0	0	0	0	0	0	0.08	0.07692308	0	0.03571429	0	0.11111111	Cluster_2
0.05555556	0	0	0.04761905	0	0	0.04166667	0.04	0.03846154	0.03703704	0	0.03448276	0.11111111	Cluster_2
0	0	0	0	0	0	0.04166667	0.04	0.03846154	0.07407407	0.03571429	0	0.11111111	Cluster_2
0.05555556	0	0	0	0	0	0	0.08	0.03846154	0.03703704	0.03571429	0.06896552	0.11111111	Cluster_2

Figura 3: Clusters 8 con sus respectivos objetos

De manera complementaria, en la Figura 4 al archivo `clusters_3.csv`, se muestra la agrupación de los mismos datos pero considerando únicamente tres clusters. En este caso, se observa una mayor concentración de objetos dentro de cada grupo, lo que implica que los clusters son más generales y abarcan una mayor cantidad de datos. Esto evidencia que al disminuir el valor de k, los grupos se vuelven más amplios y menos específicos.

1	1	1	1	1	1	1	1	1	1	1	1	0	Cluster_1
0.05555556	0	0	0	0	0.04347826	0.08333333	0.04	0.03846154	0	0.07142857	0.03448276	0.11111111	Cluster_2
0	0	0	0.04761905	0	0	0.04166667	0.04	0.03846154	0.03703704	0	0	0.11111111	Cluster_2
0.05555556	0	0	0	0	0	0.04166667	0.04	0.03846154	0.07407407	0.10714286	0.03448276	0.11111111	Cluster_2
0.05555556	0	0	0	0.09090909	0	0.08333333	0.04	0.03846154	0	0	0	0.11111111	Cluster_2
0.11111111	0	0.1	0	0.09090909	0.04347826	0.08333333	0	0.07692308	0.07407407	0.10714286	0.06896552	0.11111111	Cluster_2
0	0	0	0	0	0	0.04166667	0.08	0.07692308	0.07407407	0.03571429	0.03448276	0.33333333	Cluster_2
0.05555556	0	0	0.04761905	0	0	0.08333333	0.04	0.07692308	0.07407407	0.10714286	0.06896552	0.11111111	Cluster_2
0.05555556	0.05263158	0	0.04761905	0	0	0.08333333	0	0.07692308	0.07407407	0.07142857	0.06896552	0.11111111	Cluster_2
0.05555556	0	0	0	0	0	0.08333333	0.04	0.03846154	0.03703704	0.10714286	0.06896552	0.11111111	Cluster_2
0.05555556	0	0	0.04761905	0	0	0.04166667	0.08	0.07692308	0	0.07142857	0.06896552	0.11111111	Cluster_2
0.05555556	0	0	0.04761905	0	0	0.04166667	0.04	0.07692308	0.03703704	0.10714286	0.06896552	0.33333333	Cluster_2
0.11111111	0	0	0.04761905	0	0	0.08333333	0.08	0.03846154	0.03703704	0.03571429	0.03448276	0.44444444	Cluster_2
0	0	0	0.04761905	0	0	0.08333333	0.04	0.07692308	0.07407407	0.10714286	0.03448276	0.11111111	Cluster_2
0.05555556	0	0	0	0.04545455	0	0.08333333	0.04	0.07692308	0	0.07142857	0.03448276	0.33333333	Cluster_2
0.05555556	0	0	0	0	0	0	0.08	0.07692308	0	0.03571429	0	0.11111111	Cluster_2
0.05555556	0	0	0.04761905	0	0	0.04166667	0.04	0.03846154	0.03703704	0	0.03448276	0.11111111	Cluster_2
0	0	0	0	0	0	0.04166667	0.04	0.03846154	0.07407407	0.03571429	0	0.11111111	Cluster_2
0.05555556	0	0	0	0	0	0	0.08	0.03846154	0.03703704	0.03571429	0.06896552	0.11111111	Cluster_2

Figura 4: Clusters 3 con sus respectivos objetos

NOTA: Recordemos que normalizamos los valores y por eso salen entre 0 y 1

4.3. Matriz de confusión

Finalmente, en la Figura 5 a la matriz de confusión, se presenta la comparación entre los clusters obtenidos (columnas) y las clases reales del conjunto de datos (filas). Esta matriz permite evaluar la

calidad del agrupamiento, ya que muestra cuántos elementos de cada clase fueron asignados a cada cluster. Se observa que algunos clusters presentan valores predominantes en ciertas filas, lo que indica una buena correspondencia con una clase específica. Sin embargo, también se aprecia que **existe dispersión en algunos casos**, lo cual es esperado en métodos de agrupamiento no supervisados, donde no se utiliza información de las clases durante el proceso de entrenamiento.

```
Matriz de Confusión:
[0, 2, 6, 1, 4, 9, 6, 7]
[0, 0, 2, 0, 3, 0, 3, 0]
[0, 0, 2, 1, 1, 1, 2, 3]
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 5, 0, 0, 7, 7, 0, 2]
[0, 0, 0, 9, 0, 3, 0, 5]
[0, 0, 2, 6, 1, 2, 1, 1]
[0, 1, 4, 2, 3, 6, 5, 3]
[0, 0, 1, 2, 1, 7, 4, 2]
```

Figura 5: Matriz de Confusión generada

INTERPRETACIÓN:

Cluster	**Clase asignada**
Cluster 1	Clase 6
Cluster 2	Clase 4
Cluster 3	Clase 1
Cluster 4	Clase 7
Cluster 5	Clase 4
Cluster 6	Clase 7
Cluster 7	Clase 6
Cluster 8	Clase 1

Cuadro 1: Asignación de clusters a clases mediante la matriz de confusión

En conjunto, estas representaciones permiten analizar el comportamiento del algoritmo de Ward desde distintas perspectivas: la distribución de los datos en los clusters, el efecto del número de grupos seleccionados y la relación entre los clusters generados y las clases reales. Esto proporciona una visión completa del desempeño del método aplicado.

5. Conclusiones

A lo largo de esta práctica se logró implementar el método de Ward desde cero y aplicarlo a un conjunto de datos real, lo que permitió entender de manera más profunda cómo funcionan los algoritmos de agrupamiento.
El uso del dendrograma fue clave para visualizar cómo se van formando los grupos y para decidir un número adecuado de clusters. A partir de este análisis, se seleccionó trabajar con ocho grupos, lo cual

resultó coherente con la estructura observada en los datos.
Al revisar los resultados, se pudo notar que el algoritmo logra agrupar elementos con características similares, aunque no siempre coincide perfectamente con las clases reales. Esto es normal, ya que el método no utiliza información previa sobre las clases, sino que se basa únicamente en la similitud entre los datos.
La matriz de confusión permitió comparar los resultados obtenidos con la realidad, mostrando que algunos clusters representan bien una clase específica, mientras que otros contienen una mezcla de varias clases. Esto indica que el modelo captura parte de la estructura de los datos, pero no de forma perfecta.
En general, la práctica permitió comprender mejor el proceso de agrupamiento, desde la preparación de los datos hasta la interpretación de los resultados. Además, se observó la importancia de analizar tanto las representaciones visuales como las métricas para poder evaluar correctamente el desempeño del algoritmo.

6. APENDICES

6.1. Descripción de código

A continuación se describen las partes más importantes del código implementado para el método de Ward.

6.1.1. Lectura de datos

En esta sección se implementa la función encargada de leer el archivo CSV y separar los atributos de la clase. Figura 6

```python
# 1. LEER CSV (SIN LIBRERÍAS)
# -------------------------------
# Defino una función para leer un archivo CSV
def leer_csv(ruta):
    # Inicializo lista para guardar los datos (features)
    datos = []
    # Inicializo lista para guardar las clases (última columna)
    clases = []
    # Abro el archivo en modo lectura
    with open(ruta, 'r') as f:
        # Recorro cada línea del archivo
        for linea in f:
            # Quito espacios/saltos de línea y separo por comas
            valores = linea.strip().split(',')
            # Creo una lista para la fila actual
            fila = []
            # Recorro todos los valores excepto el último (clase)
            for i in range(len(valores)-1):
                try:
                    # Intento convertir a float
                    fila.append(float(valores[i]))
                except:
                    # Si falla, guardo 0
                    fila.append(0)
            # Agrego la fila a los datos
            datos.append(fila)
            # Guardo la última columna como clase
            clases.append(valores[-1])

    # Regreso los datos y las clases
    return datos, clases
```

Figura 6: Carga de datos

Esta función recorre cada línea del archivo, convierte los valores a tipo numérico y separa la última columna como la clase real del objeto.

6.1.2. Normalización

Se aplica normalización Min-Max para escalar los datos entre 0 y 1. Figura 7

```python
# -------------------------------
# 2. NORMALIZACIÓN (MIN-MAX)
# -------------------------------
# Defino función para normalizar los datos
def normalizar(datos):
    # Obtengo la dimensión (número de columnas)
    dim = len(datos[0])

    # Calculo el mínimo por columna
    min_vals = [min(fila[i] for fila in datos) for i in range(dim)]
    # Calculo el máximo por columna
    max_vals = [max(fila[i] for fila in datos) for i in range(dim)]

    # Lista para datos normalizados
    datos_norm = []
    # Recorro cada fila
    for fila in datos:
        nueva = []
        for i in range(dim):
            # Evito división entre cero
            if max_vals[i] - min_vals[i] == 0:
                nueva.append(0)
            else:
                # Aplico fórmula de normalización Min-Max
                nueva.append((fila[i] - min_vals[i]) / (max_vals[i] - min_vals[i]))
        # Guardo la fila normalizada
        datos_norm.append(nueva)

    # Regreso los datos normalizados
    return datos_norm
```

Figura 7: Normalización de datos

Esto es importante porque el método de Ward utiliza distancia euclidiana, la cual es sensible a la escala de los datos.

6.1.3. Distancia euclidiana

Se calcula la distancia entre dos puntos. Figura 8

```python
# -------------------------------
# 3. DISTANCIA EUCLIDIANA
# -------------------------------
# Calculo la distancia entre dos puntos
def distancia(a, b):
    suma = 0
    # Recorro cada dimensión
    for i in range(len(a)):
        # Sumo la diferencia al cuadrado
        suma += (a[i] - b[i])**2
    # Regreso la raíz cuadrada
    return suma**0.5
```

Figura 8: Calculo de distancia Euclidiana

Esta distancia es utilizada para medir la similitud entre centroides de los clusters.

6.1.4. Centroide

Se calcula el promedio de los puntos dentro de un cluster. Figura 9

```python
# -------------------------------
# 4. CENTROIDE
# -------------------------------
# Calculo el centroide de un cluster
def centroide(cluster):
    # Número de puntos
    n = len(cluster)
    # Dimensión
    dim = len(cluster[0])

    # Inicializo el centroide en cero
    c = [0]*dim
    # Sumo cada punto
    for punto in cluster:
        for i in range(dim):
            c[i] += punto[i]

    # Promedio
    for i in range(dim):
        c[i] /= n

    # Regreso el centroide
    return c
```

Figura 9: Calculo de centroides

El centroide representa el punto medio del cluster.

6.1.5. Costo de Ward

Se calcula el costo de fusionar dos clusters. Figura 10

```python
# ------------------------------
# 5. COSTO DE WARD
# ------------------------------
# Calculo el costo de unir dos clusters
def ward_cost(c1, c2):
    # Tamaños de los clusters
    n1 = len(c1)
    n2 = len(c2)

    # Obtengo centroides
    mu1 = centroide(c1)
    mu2 = centroide(c2)

    # Distancia entre centroides
    d = distancia(mu1, mu2)

    # Fórmula de Ward
    return (n1 * n2) / (n1 + n2) * (d**2)
```

Figura 10: Costo de Ward

Este valor determina qué clusters se deben unir en cada iteración.

6.1.6. Clustering jerárquico

Se implementa el algoritmo principal, donde en cada iteración se fusionan los clusters más cercanos. Figura 11

```python
# Mientras haya más de un cluster
while len(clusters) > 1:
    # Inicializo mejor par
    mejor_i, mejor_j = 0, 1
    mejor_costo = ward_cost(clusters[0], clusters[1])

    # Busco el par con menor costo
    for i in range(len(clusters)):
        for j in range(i+1, len(clusters)):
            costo = ward_cost(clusters[i], clusters[j])
            if costo < mejor_costo:
                mejor_costo = costo
                mejor_i, mejor_j = i, j

    # Obtengo IDs
    id1 = ids[mejor_i]
    id2 = ids[mejor_j]
    tamaño = len(clusters[mejor_i]) + len(clusters[mejor_j])

    # Guardo en historial
    historial.append([id1, id2, mejor_costo, tamaño])

    # Uno clusters
    nuevo = clusters[mejor_i] + clusters[mejor_j]
```

Figura 11: Clustering jerárquico

Además, se guarda un historial de fusiones para construir el dendrograma.

6.1.7. Obtención de clusters

Se generan clusters finales para un valor específico de k. Figira 12

```python
# Mientras haya más de un cluster
while len(clusters) > 1:
    # Inicializo mejor par
    mejor_i, mejor_j = 0, 1
    mejor_costo = ward_cost(clusters[0], clusters[1])

    # Busco el par con menor costo
    for i in range(len(clusters)):
        for j in range(i+1, len(clusters)):
            costo = ward_cost(clusters[i], clusters[j])
            if costo < mejor_costo:
                mejor_costo = costo
                mejor_i, mejor_j = i, j

    # Obtengo IDs
    id1 = ids[mejor_i]
    id2 = ids[mejor_j]
    tamaño = len(clusters[mejor_i]) + len(clusters[mejor_j])

    # Guardo en historial
    historial.append([id1, id2, mejor_costo, tamaño])

    # Uno clusters
    nuevo = clusters[mejor_i] + clusters[mejor_j]
```

Figura 12: Obtención de clusters

Esto permite obtener particiones como 3, 5 y 8 clusters.

6.1.8. Matriz de confusión

Se compara la asignación de clusters con las clases reales. Figura 13

```python
# ------------------------------
# 9. MATRIZ DE CONFUSIÓN
# ------------------------------
# Comparo clusters con clases reales
def matriz_confusion(clusters, clases_reales):
    # Mapeo punto a cluster
    mapa = {}
    for i, cluster in enumerate(clusters):
        for punto in cluster:
            mapa[tuple(punto)] = i

    # Predicciones
    y_pred = []
    for i in range(len(clases_reales)):
        punto = tuple(datos[i])
        y_pred.append(mapa[punto])

    # Clases únicas
    etiquetas = list(set(clases_reales))

    # Creo matriz clases x clusters
    matriz = [[0]*len(clusters) for _ in range(len(etiquetas))]

    # Lleno matriz
    for i in range(len(clases_reales)):
        fila = etiquetas.index(clases_reales[i])
        col = y_pred[i]
        matriz[fila][col] += 1

    return matriz, etiquetas
```

Figura 13: Matriz de confusión

Esto permite evaluar qué tan bien el algoritmo agrupa los datos.

6.1.9. Dendrograma

Se genera una representación visual del proceso de agrupamiento. Figura 14

```python
# ------------------------------
# 10. DENDROGRAMA
# ------------------------------
# Dibujo el dendrograma
def dibujar_dendrograma(historial, cortes=None):
    n = len(historial) + 1

    # Posiciones iniciales
    pos = {i: i for i in range(n)}
    altura = {i: 0 for i in range(n)}

    # Creo figura
    fig, ax = plt.subplots(figsize=(12, 6))

    max_dist = 0

    # Dibujo uniones
    for i, (c1, c2, dist, size) in enumerate(historial):
        x1 = pos[c1]
        x2 = pos[c2]
        y1 = altura[c1]
        y2 = altura[c2]

        ax.plot([x1, x1], [y1, dist], c='black')
        ax.plot([x2, x2], [y2, dist], c='black')
        ax.plot([x1, x2], [dist, dist], c='black')

        nuevo = n + i
        pos[nuevo] = (x1 + x2) / 2
        altura[nuevo] = dist
```

Figura 14: Dendrograma

También se incluyen líneas de corte para analizar el número de clusters.

6.1.10. Ejecución principal

Finalmente, en el bloque principal se ejecutan todas las funciones: Se generan los archivos de salida y la matriz de confusión, completando el flujo del algoritmo. Figura 15

```python
# ================================
# MAIN
# ================================

# Leo datos
datos, clases = leer_csv("DATA (1).csv")

# Normalizo
datos = normalizar(datos)

# Genero dendrograma
historial = ward_clustering(datos)
dibujar_dendrograma(historial, [0.051, 0.15, 0.30])

# Genero clusters
clusters_3 = obtener_clusters(datos, 3)
clusters_5 = obtener_clusters(datos, 5)
clusters_8 = obtener_clusters(datos, 8)

# Guardo resultados
guardar_clusters("clusters_3.csv", clusters_3)
guardar_clusters("clusters_5.csv", clusters_5)
guardar_clusters("clusters_8.csv", clusters_8)

# Matriz de confusión
matriz, etiquetas = matriz_confusion(clusters_8, clases)

# Imprimo resultados
print("\nMatriz de Confusión:")
for fila in matriz:
    print(fila)
```

Figura 15: Main

6.2. Conceptos usados

Para la implementación del método de Ward, se emplearon diversos conceptos fundamentales relacionados con el análisis de datos y el agrupamiento.
El método de Ward pertenece a la categoría de **clustering jerárquico aglomerativo**, en el cual cada objeto inicia como un cluster individual y, en cada iteración, se van fusionando los clusters más cercanos hasta formar grupos más grandes.
Para medir la similitud entre datos, se utiliza la **distancia euclidiana**, definida como:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

donde x y y son dos puntos en un espacio de n dimensiones.
Otro concepto importante es el **centroide**, que representa el promedio de los puntos dentro de un cluster:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

El método de Ward se basa en minimizar la varianza interna de los clusters. Para ello, utiliza la siguiente función de costo al unir dos clusters:

$$D(C_i, C_j) = \frac{n_i \cdot n_j}{n_i + n_j} |\mu_i - \mu_j|^2$$

Además, busca minimizar la suma de errores cuadrados (SSE), definida como:

$$SSE = \sum_{k=1}^{K} \sum_{x_i \in C_k} |x_i - \mu_k|^2$$

Finalmente, el proceso de agrupamiento se representa mediante un **dendrograma**, el cual muestra gráficamente cómo se van fusionando los clusters y permite determinar el número adecuado de grupos mediante una línea de corte.

6.3. Manual de usuario

El programa desarrollado permite aplicar el método de Ward para agrupar datos y analizar los resultados mediante dendrogramas y matrices de confusión. A continuación se describe el procedimiento para su uso.

6.3.1. 1. Preparación del entorno

Antes de ejecutar el programa, es necesario contar con Python (versión 3.x) instalado. Además, se debe instalar la librería `matplotlib`, la cual se utiliza para graficar el dendrograma. Esto se puede realizar con el siguiente comando:

```
pip install matplotlib
```

6.3.2. 2. Organización de archivos

Todos los archivos deben estar en la misma carpeta:

- `ward.py` (código principal)
- `DATA (1).csv` (base de datos)

6.3.3. 3. Ejecución del programa

Para ejecutar el programa, se debe abrir una terminal en la carpeta del proyecto y escribir:

```
python ward.py
```

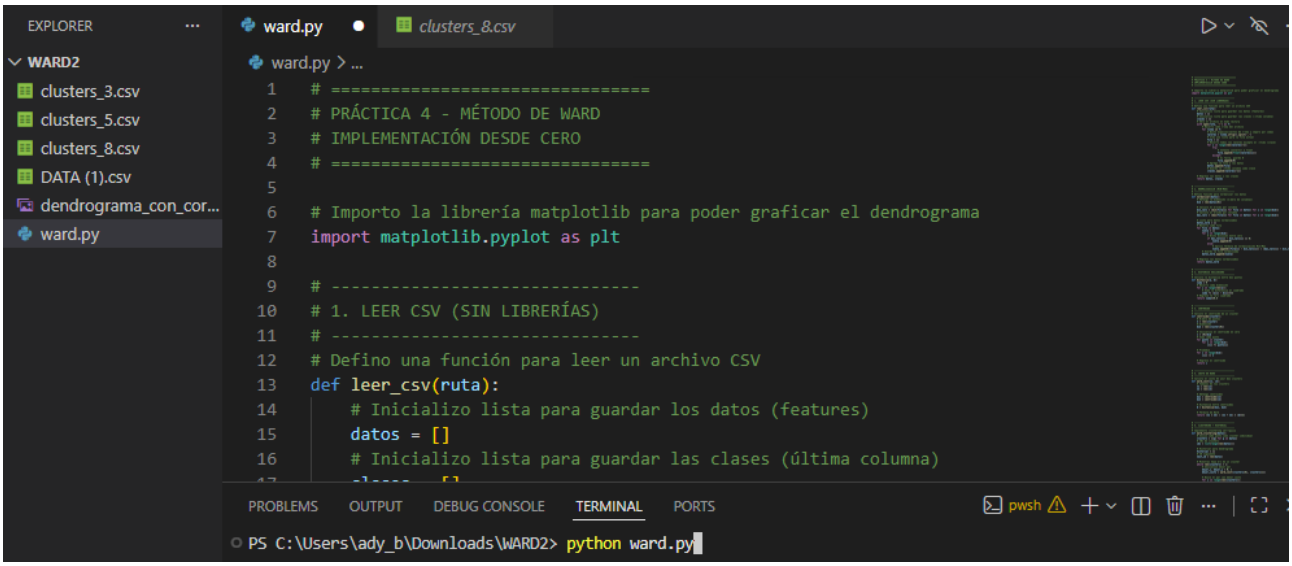

Figura 16: Ejecución

Al ejecutarse, el programa realiza automáticamente los siguientes pasos:

- Lee la base de datos.
- Normaliza los datos.
- Aplica el método de Ward.
- Genera el dendrograma.

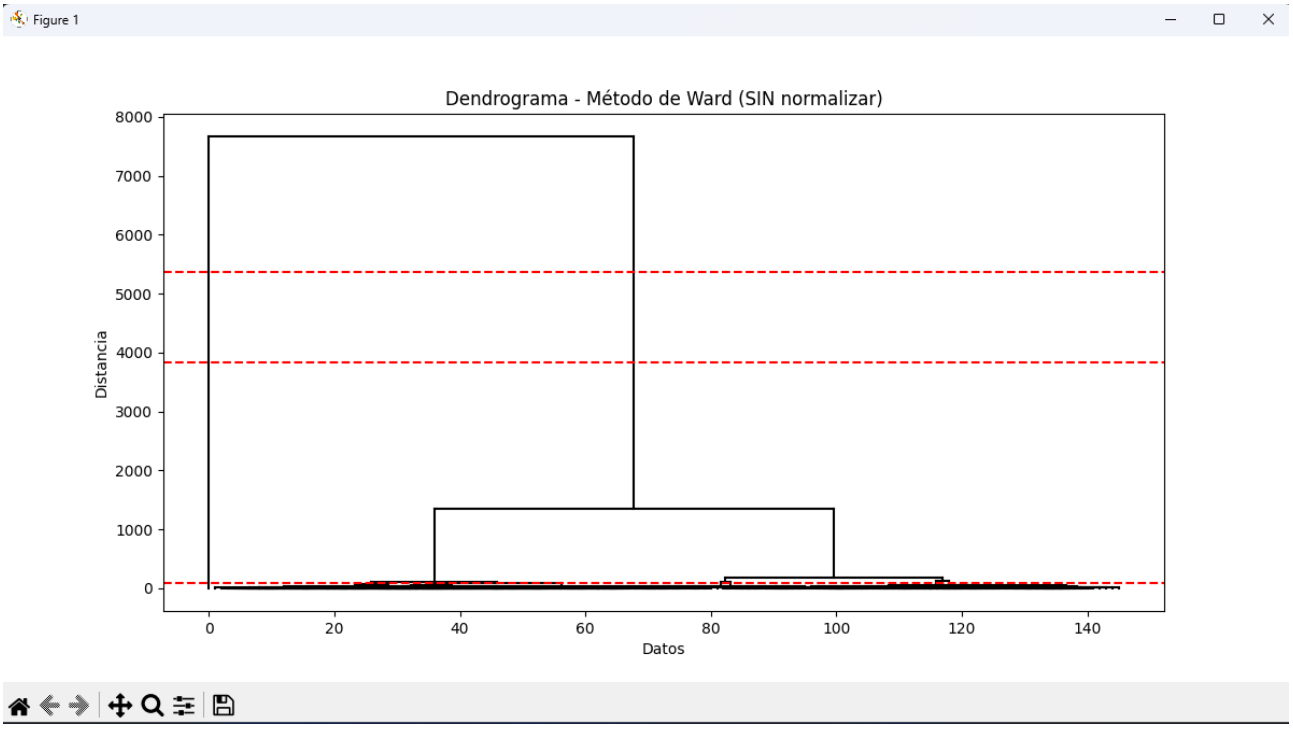

Figura 17: Ventana desplegable del Dendrograma, cerrar para continuar ejecución

- Calcula clusters para $k = 3$, $k = 5$ y $k = 8$.
- Genera la matriz de confusión.

6.3.4. 4. Resultados generados

Después de la ejecución, se generan los siguientes archivos:

- `clusters_3.csv`
- `clusters_5.csv`
- `clusters_8.csv`

Cada archivo contiene los datos originales junto con el cluster asignado a cada objeto. Figura 18
Además, se genera una imagen del dendrograma. Figura 18

- `dendrograma_con_cortes.png`

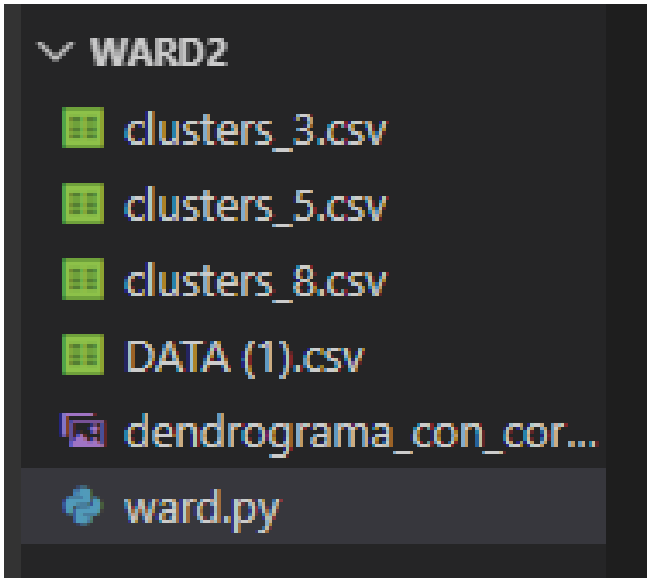

Figura 18: Archivos generados

6.3.5. 5. Interpretación de resultados

En los archivos CSV, cada fila corresponde a un objeto del dataset y la última columna indica el cluster al que pertenece. Esto permite analizar cómo se agrupan los datos.
En la consola se muestra la matriz de confusión, la cual permite comparar los clusters generados con las clases reales del dataset. Esta información es utilizada para evaluar el desempeño del algoritmo. Figura 19

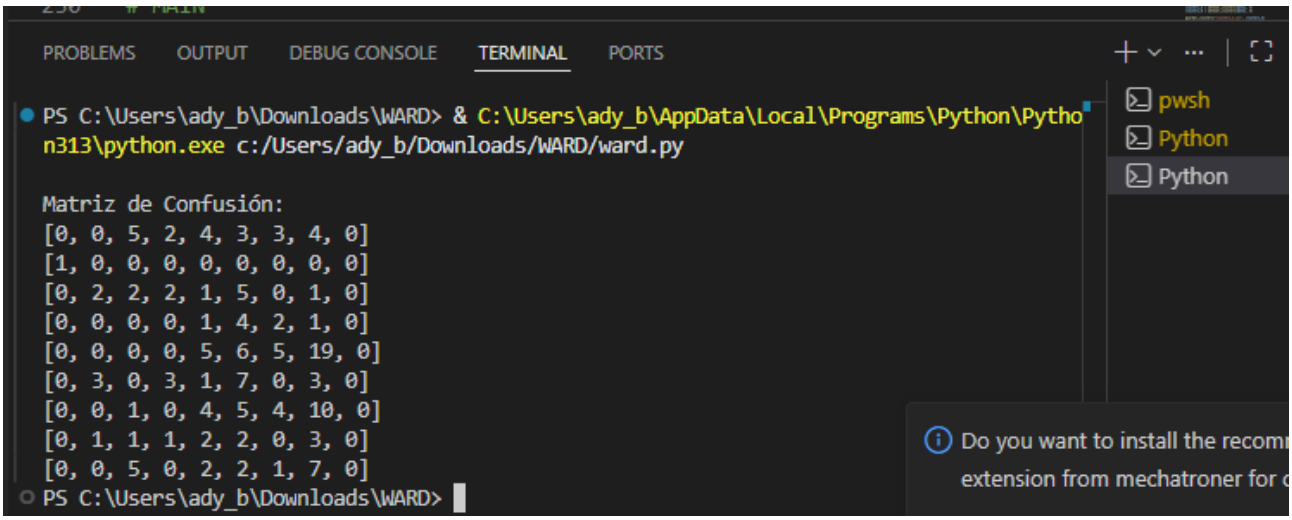

Figura 19: Matriz de confusión desde terminal

6.3.6. 6. Recomendaciones

Se recomienda verificar que el archivo CSV tenga el formato correcto (última columna como clase) y que no existan errores en la ruta del archivo. También es importante mantener todos los archivos en la misma carpeta para evitar problemas de ejecución.

6.4. Manual técnico

El sistema fue desarrollado en el lenguaje de programación Python, utilizando una implementación propia del método de Ward sin el uso de librerías especializadas de aprendizaje automático. Esto permitió tener un control completo sobre cada etapa del algoritmo.
Para la ejecución del programa es necesario contar con Python instalado (versión 3.x). La única librería externa utilizada fue `matplotlib`, la cual se emplea para la generación y visualización del dendrograma. Esta puede instalarse mediante el comando:

```
pip install matplotlib
```

El programa se ejecuta desde la terminal o consola mediante el siguiente comando:

```
python ward.py
```

Es importante que el archivo de datos (por ejemplo, `DATA (1).csv`) se encuentre en la misma carpeta que el script, ya que el programa realiza la lectura directa desde este archivo. El formato del archivo debe ser CSV, donde las primeras columnas corresponden a los atributos numéricos y la última columna representa la clase real.
Al ejecutar el programa, se generan los siguientes resultados:

- Un dendrograma que muestra el proceso de agrupamiento.
- Tres archivos CSV (`clusters_3.csv`, `clusters_5.csv`, `clusters_8.csv`) con la asignación de cada dato a su cluster.
- La matriz de confusión impresa en consola.

Para visualizar correctamente el dendrograma, se requiere una interfaz gráfica, ya que se abre en una ventana interactiva que permite hacer zoom y analizar los cortes.
En caso de errores, se recomienda verificar la ruta del archivo CSV, el formato de los datos y la correcta instalación de la librería utilizada.

6.5. Referencias

Referencias

[1] Yilmaz, N., Sekeroglu, B. (2019). *Higher Education Students Performance Evaluation.* UCI Machine Learning Repository. Disponible en: https://archive.ics.uci.edu/dataset/856/higher+education+students+performance+evaluation